

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования

## **ОТЧЁТ**

по дисциплине «Разработка мобильных приложений»

**Мобильное приложение для трейдинга и инвестиций  
с микросервисным бэкендом на 10 000 одновременных клиентов**

Выполнил: студент  
Лашкул А. В.

Проверил:

# СОДЕРЖАНИЕ

|     |                                                |    |
|-----|------------------------------------------------|----|
| 1   | Введение .....                                 | 3  |
| 2   | Аналитический раздел .....                     | 4  |
| 2.1 | Обзор существующих решений .....               | 4  |
| 2.2 | Обоснование выбора технологий .....            | 4  |
| 3   | Проектный раздел .....                         | 5  |
| 3.1 | Архитектура системы .....                      | 5  |
| 3.2 | Драйвер ядра (7) .....                         | 5  |
| 3.3 | Сервис котировок (6) .....                     | 5  |
| 3.4 | Data-сервис (4) .....                          | 6  |
| 3.5 | Gateway (3) .....                              | 6  |
| 3.6 | Android-приложение (1) .....                   | 6  |
| 3.7 | Схема базы данных .....                        | 6  |
| 4   | Реализация .....                               | 8  |
| 5   | Тестирование .....                             | 9  |
| 5.1 | Юнит- и интеграционные тесты .....             | 9  |
| 5.2 | Нагрузочное тестирование 10 000 клиентов ..... | 9  |
| 6   | Заключение .....                               | 10 |
|     | Список литературы .....                        | 11 |

# 1 ВВЕДЕНИЕ

Мобильные инвестиционные приложения стали массовым продуктом: брокерские платформы обслуживают миллионы частных инвесторов, и ключевыми требованиями к ним являются доставка рыночных данных в реальном времени, корректность исполнения торговых операций и устойчивость к высокой одновременной нагрузке.

**Цель работы** — спроектировать и реализовать полную экосистему биржевого терминала: от источника котировок на уровне ядра операционной системы до нативного мобильного клиента, способную обслуживать 10 000 одновременных клиентов.

## **Задачи работы:**

1. реализовать драйвер ядра Linux, имитирующий поток биржевых котировок;
2. реализовать микросервис сбора котировок с публикацией в брокер сообщений и записью истории в аналитическую СУБД;
3. реализовать сервис работы с базой данных: счета, ордера, сделки, портфели, матчинг лимитных заявок;
4. реализовать gateway-сервис для мобильных клиентов: аутентификация, потоковая раздача котировок, проксирование торговых операций;
5. разработать нативное Android-приложение (Kotlin, Jetpack Compose);
6. провести нагрузочное тестирование на 10 000 одновременных клиентов;
7. обеспечить наблюдаемость системы (распределённые трейсы, метрики, логи).

## 2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

### 2.1 Обзор существующих решений

Современные брокерские приложения (Тинькофф Инвестиции, СберИнвестор, Robinhood) построены по схожей схеме: тонкий мобильный клиент, единая точка входа (API-gateway), потоковая доставка котировок по WebSocket и отдельные сервисы исполнения ордеров и учёта позиций. Историческое хранение рыночных данных, как правило, вынесено в колоночные СУБД, оптимизированные под время-ряды.

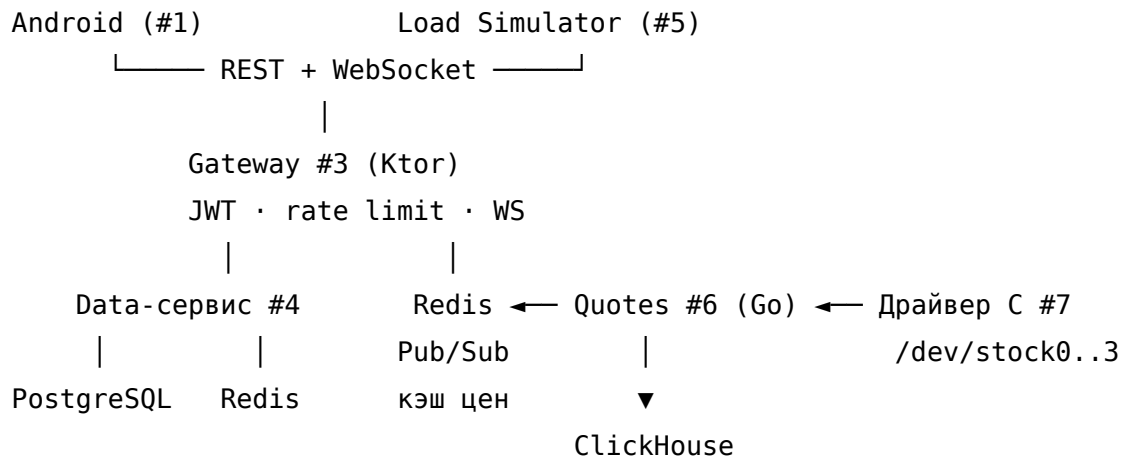
### 2.2 Обоснование выбора технологий

| Слой           | Выбор и обоснование                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------|
| Драйвер        | C, модуль ядра Linux — имитация источника котировок на максимально низком уровне; модель геометрического броуновского движения |
| Сбор котировок | Go — дешёвые горутины для блокирующего чтения символьных устройств, простой HTTP/SSE                                           |
| Бэкенд         | Kotlin + Ktor — корутины, единый язык с Android-клиентом; голый SQL без ORM (требование задания)                               |
| Брокер/кэш     | Redis — Pub/Sub для тиков и кэш последних цен                                                                                  |
| СУБД           | PostgreSQL — транзакционная целостность счетов и сделок                                                                        |
| История        | ClickHouse — колоночное хранение тиков, свечи через материализованные представления                                            |
| Клиент         | Kotlin + Jetpack Compose, MVVM, StateFlow                                                                                      |
| Наблюдаемость  | OpenTelemetry + Jaeger — сквозные трейсы                                                                                       |

## 3 ПРОЕКТНЫЙ РАЗДЕЛ

### 3.1 Архитектура системы

Система состоит из семи компонентов (рис. 1): драйвер ядра (7) генерирует тики; Go-сервис (6) читает их, публикует в Redis и пишет в ClickHouse; Data-сервис (4) ведёт счета и исполняет ордера; Gateway (3) обслуживает клиентов — Android-приложение (1) и нагрузочный имитатор (5).



Листинг 1. Архитектура системы

**Поток котировок:** драйвер → quotes-go → Redis Pub/Sub (quotes:ticks) + ClickHouse (батчи) → Gateway (одна подписка, fan-out по WebSocket) → клиенты.

**Поток торговли:** клиент → Gateway (JWT, userId из токена) → Data-сервис: market исполняется по цене из Redis-кэша; limit матчится фоновым воркером по потоку тиков. Сделка, позиция и баланс изменяются в одной транзакции.

### 3.2 Драйвер ядра (7)

Модуль stock\_gen.c создаёт символьные устройства /dev/stock0..3. Цена моделируется геометрическим броуновским движением. Запись — 32 байта (little-endian): seq u64, timestamp\_ns s64, price\_udollar s64, volume u32, dev\_idx u32. Поддерживаются блокирующее чтение, poll и ioctl (установка базовой цены, волатильности, дрефта, интервала тика).

### 3.3 Сервис котировок (6)

Go-сервис читает устройства (или генерирует тики встроенным GBM-симулятором, когда /dev/stock\\* недоступны), маппит индекс устройства на тикер

(SBER, GAZP, AAPL, BTC) и раздаёт данные потребителям: REST/SSE для отладки, Redis Pub/Sub `quotes:ticks` + ключи `quote:last:<symbol>` для остальных сервисов, батч-запись в ClickHouse (раз в секунду или по 5000 строк). Метрики (тиков/сек, ошибки записи) экспонируются в формате Prometheus.

### 3.4 Data-сервис (4)

Ядро бизнес-логики на Ktor. Работа с PostgreSQL — голый SQL через JDBC + HikariCP: репозитории принимают `Connection`, поэтому операции komponуются в одну транзакцию. Только параметризованные запросы.

Исполнение ордера: блокировка строки счёта (`SELECT ... FOR UPDATE`), проверка средств или позиции, запись сделки, пересчёт средней цены позиции и баланса — атомарно. Недостаток средств — статус `REJECTED`. Идемпотентность обеспечивает уникальный `client_order_id`. Лимитные заявки исполняет фоновый воркер, слушающий `quotes:ticks`: пересечённые лимитки выбирают-ся запросом с `FOR UPDATE SKIP LOCKED` и исполняются по цене тика.

### 3.5 Gateway (3)

Единая точка входа: регистрация и вход (bcrypt + JWT HS256, access 15 мин / refresh 7 дней), список котировок с изменением за день, история свечей из ClickHouse (агрегация тиков на леты), WebSocket-стрим с подпиской по символам (одна подписка на Redis на весь процесс, fan-out через SharedFlow), проксирование торговых маршрутов в Data-сервис с подстановкой `userId` из JWT и rate limiting на IP.

### 3.6 Android-приложение (1)

Kotlin + Jetpack Compose, архитектура MVVM (ViewModel + StateFlow), ручной DI. Экраны: авторизация, рынок (живые цены по WebSocket), карточка инструмента с графиком (Canvas) и формой ордера Market/Limit, портфель с P&L, история ордеров и сделок с отменой активных лимиток. JWT хранится в DataStore; при 401 клиент автоматически обновляет access-токен и повторяет запрос.

### 3.7 Схема базы данных

```
users      (id, login UNIQUE, password_hash, created_at)
accounts   (id, user_id FK, currency, cash_balance CHECK >= 0)
instruments(id, symbol UNIQUE, name, dev_idx)
```

```
orders      (id, user_id, symbol, side, type, qty, price, filled_qty,
             status, client_order_id UNIQUE, created_at, updated_at)
trades      (id, order_id, user_id, symbol, side, qty, price, fee,
             executed_at)
positions   (user_id, symbol, qty CHECK >= 0, avg_price, PK(user_id, symbol))

ClickHouse: ticks (symbol, ts, price, volume, seq) ENGINE=MergeTree,
свечи candles_1m — AggregatingMergeTree, заполняется материализованным
представлением.
```

## 4 РЕАЛИЗАЦИЯ

Ключевые решения подробно описаны в проектном разделе; отметим технологические детали:

- **Redis Pub/Sub как шина тиков** развязывает генерацию котировок и потребителей: Gateway и Data-сервис подписаны независимо, отказ одного не влияет на другой.
- **Денежные поля в JSON передаются строками** — `NUMERIC(18,6)` не представим в `double` без потери точности.
- **Сквозной трейсинг**: Gateway инжектирует W3C traceparent в запросы к Data-сервису; спаны SQL-транзакций становятся детьми клиентского запроса. Контекст переносится между корутинами через `opentelemetry-extension-kotlin`.
- **Защита от перегрузки**: rate limiting в Gateway, неблокирующие буферы в каналах тиков (медленный потребитель теряет старые тики, но не останавливает систему).



## 5 ТЕСТИРОВАНИЕ

### 5.1 Юнит- и интеграционные тесты

В каждом сервисе — юнит-тесты бизнес-логики (маппинг символов, кодирование батчей, средняя цена позиции, пересечение лимиток, JWT, перцентили). Интеграционный сценарий проверен на реальном окружении (PostgreSQL, Redis, ClickHouse): регистрация → market-ордер → сделка → портфель; лимитная заявка исполняется воркером при достижении цены; повторный запрос с тем же `clientOrderId` возвращает существующий ордер.

### 5.2 Нагрузочное тестирование 10 000 клиентов

Имитатор поднимает 10 000 виртуальных клиентов (горутины), каждый регистрируется и выполняет случайные действия с заданными весами; 1 000 клиентов дополнительно держат WebSocket-подписку. Плавный ramp-up 60 с, рабочая фаза 90 с, 2 000 запросов/с.

| Действие  | Запросов | Ошибок | p50, мс | p95, мс | p99, мс | max, мс |
|-----------|----------|--------|---------|---------|---------|---------|
| quotes    | 110 119  | 0      | 10.8    | 71.9    | 120.0   | 316.6   |
| portfolio | 49 257   | 0      | 11.9    | 73.9    | 122.9   | 317.9   |
| order     | 36 987   | 5      | 22.9    | 91.3    | 143.6   | 322.9   |
| trades    | 24 543   | 0      | 11.1    | 71.5    | 119.6   | 264.1   |
| history   | 24 277   | 5      | 30.4    | 126.9   | 209.4   | 506.2   |
| register  | 10 000   | 0      | 12.6    | 52.2    | 121.9   | 402.2   |

Таблица 1. Результаты нагрузочного теста на 10 000 клиентов

Итого 255 193 запроса, 10 ошибок (0{,}004 %). WebSocket: 1 000 стабильных соединений, 6{,}08 млн доставленных тиков, ни одного переподключения.

## 6 ЗАКЛЮЧЕНИЕ

В ходе работы реализована полная экосистема биржевого терминала:

1. драйвер ядра Linux с имитацией котировок (GBM);
2. Go-сервис сбора котировок с Redis Pub/Sub и записью в ClickHouse;
3. Data-сервис на Ktor с голым SQL: счета, ордера, сделки, матчинг лимиток;
4. Gateway с JWT-аутентификацией, WebSocket-стримом и rate limiting;
5. Android-приложение на Jetpack Compose (MVVM);
6. нагрузочный имитатор, подтвердивший обслуживание 10 000 одновременных клиентов с  $p_{99} \leq 210$  мс и долей ошибок 0{,}004 %;
7. наблюдаемость: сквозные трейсы OpenTelemetry, метрики, структурированные логи; документация Dokka и Obsidian.

Направления развития: частичное исполнение ордеров (PARTIALLY\_FILLED), стакан заявок и матчинг «клиент против клиента», React Native клиент, горизонтальное масштабирование Gateway за балансировщиком.

## СПИСОК ЛИТЕРАТУРЫ

1. Love R. Linux Kernel Development. — 3-е изд. — Addison-Wesley, 2010.
2. Donovan A., Kernighan B. The Go Programming Language. — Addison-Wesley, 2015.
3. Ktor Documentation. — URL: <https://ktor.io/docs/> (дата обращения: 12.06.2026).
4. Jetpack Compose Documentation. — URL: <https://developer.android.com/compose> (дата обращения: 12.06.2026).
5. Redis Pub/Sub. — URL: <https://redis.io/docs/latest/develop/interact/pubsub/> (дата обращения: 12.06.2026).
6. ClickHouse Documentation. — URL: <https://clickhouse.com/docs> (дата обращения: 12.06.2026).
7. OpenTelemetry Specification. — URL: <https://opentelemetry.io/docs/specs/otel/> (дата обращения: 12.06.2026).
8. Hull J. Options, Futures, and Other Derivatives. — 11-е изд. — Pearson, 2021.